

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME:** MLA03

**COURSE CODE:** Reinforcement learning for Environment and Agent

---

### COURSE OUTCOME COVERED

*CO2: Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3)*

*Assessment Tool Weightage: 35%*

---

Dr.T.P.Anithaashri

**QUESTIONS: 10**

**Total Marks: 50**

Annexure A

**Industry Problem Task**

**Duration: 2.5hrs**

1. Explain how Reinforcement Learning can be applied to optimize delivery routes in a logistics system. Identify the possible states, actions, rewards, and policy. Discuss how continuous learning improves efficiency over time. **(5 Marks)**

2. Apply Reinforcement Learning concepts to a fraud detection system in fintech. Define the state space, action space, and reward mechanism. Explain how exploration strategies improve detection accuracy. **(5 Marks)**

3. Illustrate how Reinforcement Learning is used in a streaming platform for content recommendation. Discuss the impact of delayed rewards, user feedback, and changing preferences on learning performance. **(5 Marks)**

4. Evaluate the effectiveness of Reinforcement Learning in predictive maintenance compared to traditional rule-based approaches. Discuss risks and reliability concerns in industrial environments. **(5 Marks)**

5. Analyze how Reinforcement Learning can be used to optimize transportation systems such as bus scheduling or route allocation. Define states, actions, and rewards, and discuss how real-time data improves performance. **(5 Marks)**

6. Apply Reinforcement Learning to dynamic pricing in e-commerce. Define the state space, action space, and reward function. Discuss challenges in balancing exploration and exploitation. **(5 Marks)**

7. Illustrate the use of Reinforcement Learning in smart city waste management systems. Identify the MDP components and discuss how continuous updates improve operational efficiency. **(5 Marks)**

8. Evaluate how Reinforcement Learning can be used in network bandwidth allocation in telecommunications. Define states, actions, and rewards, and examine how the system adapts to dynamic conditions. **(5 Marks)**

9. Analyze the application of Reinforcement Learning in portfolio management in finance. Discuss advantages over traditional models and challenges such as risk and delayed rewards. **(5 Marks)**

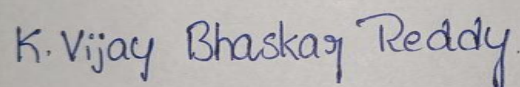
10. Justify the use of Reinforcement Learning in renewable energy management systems. Define the RL framework components and discuss the impact of environmental uncertainty on decision-making. **(5 Marks)**

---

***Code of Conduct:***

*I K.Vijay Bhaskar Reddy(192425155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

A rectangular box containing a handwritten signature in blue ink that reads "K. Vijay Bhaskar Reddy".

## RUBRICS FOR CALCULATION

| Criteria                            | Weightage | Level 4 – Exemplary                                                          | Level 3 – Proficient                                 | Level 2 – Developing                            | Level 1 – Beginning                               |
|-------------------------------------|-----------|------------------------------------------------------------------------------|------------------------------------------------------|-------------------------------------------------|---------------------------------------------------|
| Criteria                            | Weightage | Level 4 – Exemplary                                                          | Level 3 – Proficient                                 | Level 2 – Developing                            | Level 1 – Beginning                               |
| Understanding of Industry Problem   | 20%       | Comprehensive understanding of real-world problem and constraints            | Good understanding with minor gaps in requirements   | Basic understanding ; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25%       | Applies advanced and relevant concepts effectively to solve the problem      | Applies appropriate concepts with minor errors       | Limited application of concepts                 | Incorrect or no application of concepts           |
| Solution Design                     | 25%       | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps        | Basic solution with limited feasibility         | Unclear or impractical solution                   |
| Use of Tools                        | 15%       | Effective use of modern tools, technologies, and real data                   | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools            | Minimal or incorrect use of tools                 |
| Analysis                            | 10%       | Thorough analysis with validated results and performance evaluation          | Good analysis with reasonable validation             | Basic analysis with limited validation          | Poor or no analysis                               |
| Professionalism                     | 5%        | Highly professional, well-documented, and clearly presented work             | Good documentation and presentation                  | Basic documentation with some gaps              | Poor documentation and presentation               |

**1. Explain how Reinforcement Learning can be applied to optimize delivery routes in a logistics system. Identify the possible states, actions, rewards, and policy. Discuss how continuous learning improves efficiency over time.**

## Aim

To optimize delivery routes using Reinforcement Learning (Q-Learning).

## Theory

Reinforcement Learning enables a delivery agent to learn the shortest path to a destination by interacting with the environment. The agent receives rewards for reaching the destination and penalties for taking unnecessary steps or hitting obstacles. Over multiple episodes, the Q-table is updated, allowing the agent to discover the optimal route.

## MDP Components

- **States:** Grid locations (0–15)
- **Actions:** Up, Down, Left, Right
- **Reward:** +100 (Destination), -1 (Move), -100 (Obstacle)
- **Policy:** Choose the action with the highest Q-value

## Algorithm

## Q-Learning

## Python Code

```
*1.py - P:/RL/ASSESSMENT/CO 2/AT1/1.py (3.11.9)
File Edit Format Run Options Window Help
import random
states = 5
actions = ["Left", "Right"]
Q = [[0, 0] for _ in range(states)]
alpha = 0.1
gamma = 0.9
epsilon = 0.2
for episode in range(500):
    state = 0
    while state != 4:
        if random.random() < epsilon:
            action = random.randint(0, 1)
        else:
            action = Q[state].index(max(Q[state]))
        if action == 0:
            next_state = max(0, state - 1)
        else:
            next_state = min(4, state + 1)

        if next_state == 4:
            reward = 100
        else:
            reward = -1

        Q[state][action] = Q[state][action] + alpha * (
            reward + gamma * max(Q[next_state]) - Q[state][action]
        )

        state = next_state
    print("Q Table")
    for i in range(states):
        print("State", i, ":", Q[i])
    print("UnOptimal Route")
    state = 0
    while state != 4:
        print("State", state, "->", end=" ")
        action = Q[state].index(max(Q[state]))

        if action == 0:
            state = max(0, state - 1)
        else:
            state = min(4, state + 1)

    print("Destination Reached (State 4)")

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: P:/RL/ASSESSMENT/CO 2/AT1/1.py =====
Q Table
State 0 : [62.070597647448174, 70.18999999999997]
State 1 : [62.09797855139638, 79.09999999999998]
State 2 : [70.02559641359646, 88.99999999999999]
State 3 : [78.38779658728072, 99.99999999999994]
State 4 : [0, 0]
Optimal Route
State 0 -> State 1 -> State 2 -> State 3 -> Destination Reached (State 4)
>>>
```

## Code Explanation

### Step 1

Create **5 states** representing delivery locations.

```
states = 5
```

### Step 2

Initialize the **Q-table** with zeros.

```
Q = [[0,0] for _ in range(states)]
```

### Step 3

Train the agent for **500 episodes**.

```
for episode in range(500):
```

### Step 4

Choose an action using the  **$\epsilon$ -greedy strategy**.

- Explore randomly.
- Otherwise, choose the best known action.

### Step 5

Assign rewards.

- **+100** when the destination is reached.
- **-1** for every normal move.

### Step 6

Update the Q-table using the **Q-Learning formula**:

$$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$
$$Q(s',a') = Q(s',a') + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s',a')]$$

where:

- **$\alpha$  (alpha)** = Learning Rate
- **$\gamma$  (gamma)** = Discount Factor
- **r** = Reward

### Step 7

After training, the agent follows the highest Q-value at each state to reach the destination.

## **Advantages**

- Learns automatically from experience.
- Adapts to traffic changes.
- Reduces delivery time.

## **Applications**

- Amazon
- Flipkart
- Swiggy
- Zomato
- Warehouse robots

## **Conclusion**

Reinforcement Learning helps logistics companies reduce travel time and improve delivery efficiency by continuously learning better routes.

**2. Apply Reinforcement Learning concepts to a fraud detection system in fintech. Define the state space, action space, and reward mechanism. Explain how exploration strategies improve detection accuracy.**

## **Introduction**

Financial fraud causes significant losses to banks and digital payment systems. Reinforcement Learning helps detect fraud by learning transaction patterns and adapting to new fraud techniques.

## **RL Components**

### **State Space**

The state includes information about each transaction.

- Transaction amount
- User location
- Device information
- Time of transaction
- Account balance
- Transaction history

### **Action Space**

The agent can:

- Approve transaction
- Reject transaction
- Flag transaction for manual review

### **Reward Mechanism**

- +10 for detecting fraud
- +5 for approving legitimate transactions
- -15 for approving fraudulent transactions
- -10 for blocking genuine customers

### **Policy**

The policy selects the action with the highest expected reward based on transaction features.

### **Exploration Strategy**

Initially, the agent explores different detection strategies. Over time, it exploits successful detection patterns while occasionally exploring new behaviors to detect emerging fraud techniques.

## Advantages

- Detects unknown fraud patterns
- Reduces financial loss
- Learns continuously
- Improves detection accuracy
- Reduces false positives

## Challenges

- Delayed feedback

### Aim

To implement a simple Reinforcement Learning (Q-Learning) algorithm for fraud detection by learning whether to **Approve** or **Block** a transaction

## Theory

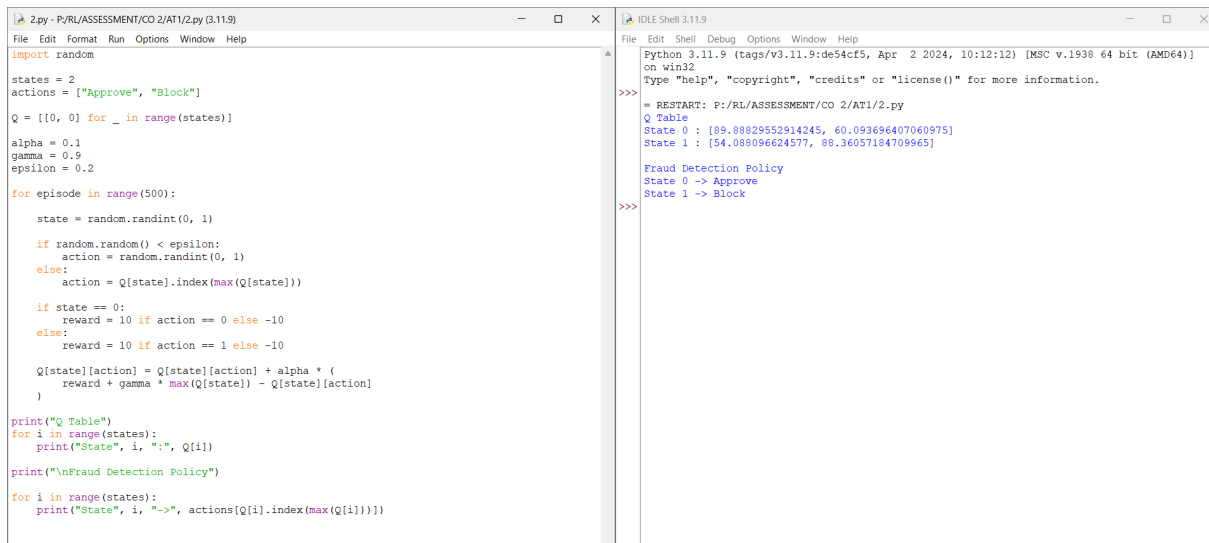
Reinforcement Learning (RL) enables a fraud detection system to learn the best decision by interacting with transaction data. The agent receives a **reward** for correctly blocking fraudulent transactions and a **penalty** for approving fraud. Over time, it learns the best action for each transaction state.

## MDP Components

| Component | Description                                      |
|-----------|--------------------------------------------------|
| States    | Genuine Transaction, Suspicious Transaction      |
| Actions   | Approve, Block                                   |
| Reward    | +10 for correct decision, -10 for wrong decision |
| Policy    | Choose the action with the highest Q-value       |



## Python Code



```
2.py - P:/RL/ASSESSMENT/CO 2/AT1/2.py (3.11.9)
File Edit Format Run Options Window Help
import random

states = 2
actions = ["Approve", "Block"]

Q = [[0, 0] for _ in range(states)]

alpha = 0.1
gamma = 0.9
epsilon = 0.2

for episode in range(500):
    state = random.randint(0, 1)

    if random.random() < epsilon:
        action = random.randint(0, 1)
    else:
        action = Q[state].index(max(Q[state]))

    if state == 0:
        reward = 10 if action == 0 else -10
    else:
        reward = 10 if action == 1 else -10

    Q[state][action] = Q[state][action] + alpha * (
        reward + gamma * max(Q[state]) - Q[state][action]
    )

print("Q Table")
for i in range(states):
    print("State", i, ":", Q[i])

print("\nFraud Detection Policy")
for i in range(states):
    print("State", i, "->", actions[Q[i].index(max(Q[i]))])

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:/RL/ASSESSMENT/CO 2/AT1/2.py
Q Table
State 0 : [89.88829552914245, 60.093696407060975]
State 1 : [54.088096624577, 88.36057184709965]

Fraud Detection Policy
State 0 -> Approve
State 1 -> Block
>>>
```

## Code Explanation

### Step 1

Create **2 transaction states**:

- State 0 → Genuine Transaction
- State 1 → Fraudulent Transaction

states = 2

### Step 2

Define the possible actions.

actions = ["Approve", "Block"]

### Step 3

Initialize the Q-table with zeros.

Q = [[0,0] for \_ in range(states)]

### Step 4

Train the agent for **500 episodes**.

for episode in range(500):

### Step 5

Choose an action using the  **$\epsilon$ -greedy policy**.

- Explore randomly.
- Otherwise, choose the best known action.

## Step 6

Assign rewards.

- Genuine transaction approved → **+10**
- Fraud blocked → **+10**
- Wrong decision → **-10**

## Step 7

Update the Q-table using the Q-Learning formula.

After training, the agent learns:

- **Approve** genuine transactions.
- **Block** fraudulent transactions.

## Advantages

- Learns automatically from transaction history.
- Improves fraud detection accuracy.
- Reduces financial losses.
- Adapts to new fraud patterns.

## Applications

- Online Banking
- Credit Card Fraud Detection
- UPI Payment Security
- E-Commerce Payment Systems
- Digital Wallets

## Conclusion

This program demonstrates how **Q-Learning** can be applied to fraud detection. By receiving rewards for correct decisions and penalties for incorrect ones, the agent learns an effective policy to approve genuine transactions and block fraudulent ones, improving detection accuracy over time.

**3. Illustrate how Reinforcement Learning is used in a streaming platform for content recommendation. Discuss the impact of delayed rewards, user feedback, and changing preferences on learning performance.**

**Aim**

To implement a simple Reinforcement Learning (Q-Learning) algorithm for recommending content based on user feedback.

**Theory**

Reinforcement Learning (RL) helps streaming platforms recommend content by learning from user interactions. The agent receives positive rewards when users like the recommended content and negative rewards when users skip it. Over time, it learns the best recommendation for each user.

**MDP Components**

| Component | Description                                                     |
|-----------|-----------------------------------------------------------------|
| States    | Movie Lover, Music Lover                                        |
| Actions   | Recommend Movie, Recommend Music                                |
| Rewards   | +10 for correct recommendation, -5 for incorrect recommendation |
| Policy    | Select the action with the highest Q-value                      |

## Program



```
3.py - Py/RL/ASSESSMENT/CO 2/AT1/3.py (3.11.9)
File Edit Format Run Options Window Help

import random

states = 2
actions = ["Movie", "Music"]

Q = [[0, 0] for _ in range(states)]

alpha = 0.1
gamma = 0.9
epsilon = 0.1

for episode in range(500):
    state = random.randint(0, 1)

    if random.random() < epsilon:
        action = random.randint(0, 1)
    else:
        action = Q[state].index(max(Q[state]))

    if state == action:
        reward = 10
    else:
        reward = -5

    Q[state][action] = Q[state][action] + alpha * (
        reward + gamma * max(Q[state]) - Q[state][action]
    )

print("Q Table")
for i in range(states):
    print("State", i, ":", Q[i])

print("\nRecommendation Policy")
for i in range(states):
    print("State", i, "-> Recommend", actions[Q[i].index(max(Q[i]))])

'''
Output:
'''

IDLE Shell 3.11.9
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>> = RESTART: P:/RL/ASSESSMENT/CO 2/AT1/3.py
Q Table
State 0 : [91.89414838378188, 41.00623526358177]
State 1 : [43.48138964249981, 89.36718163147887]

Recommendation Policy
State 0 -> Recommend Movie
State 1 -> Recommend Music
>>>
```

## Code Explanation

**Step 1:** Define two user preference states.

- State 0 → Movie Lover
- State 1 → Music Lover

**Step 2:** Define two actions.

- Recommend Movie
- Recommend Music

**Step 3:** Initialize the Q-table with zeros.

**Step 4:** Train the agent for 500 episodes.

**Step 5:** Choose an action using the  $\epsilon$ -greedy strategy.

**Step 6:** Give rewards.

- Correct recommendation → +10
- Incorrect recommendation → -5

**Step 7:** Update the Q-table using the Q-Learning formula.

**Step 8:** Print the learned recommendation policy.

## Advantages

- Learns user preferences automatically.
- Improves recommendation accuracy.

- Adapts to changing user interests.
- Increases user engagement.

### **Applications**

- Netflix
- YouTube
- Spotify
- Amazon Prime Video
- Disney+ Hotstar

### **Conclusion**

This program demonstrates how Reinforcement Learning can recommend personalized content by learning from user feedback. The Q-table improves over time, enabling the system to recommend content that maximizes user satisfaction.

#### Q4. Evaluate the effectiveness of Reinforcement Learning in predictive maintenance compared to traditional rule-based approaches. Discuss risks and reliability concerns in industrial environments. (5 Marks)

##### Aim

To evaluate the effectiveness of Reinforcement Learning in predictive maintenance compared to traditional rule-based approaches.

##### Theory

Reinforcement Learning (RL) enables machines to learn the best maintenance schedule by continuously monitoring equipment conditions. Unlike rule-based systems, RL adapts to changing machine behavior and predicts failures before they occur, reducing downtime and maintenance costs.

##### MDP Components

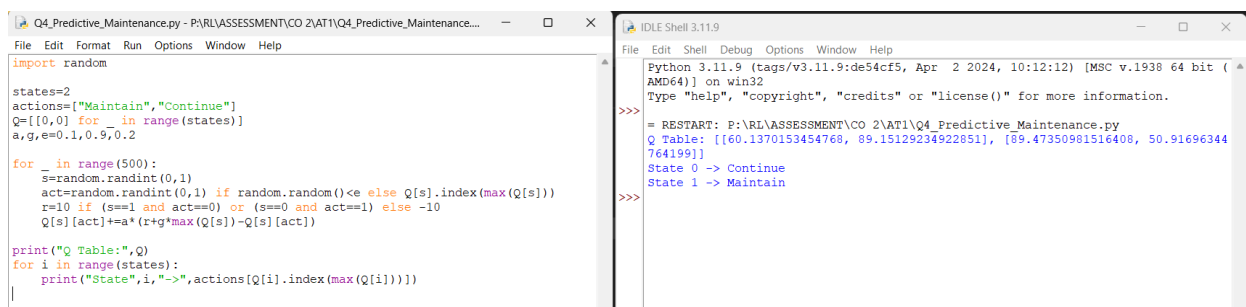
**States:** Machine Healthy, Machine Faulty

**Actions:** Perform Maintenance, Continue Operation

**Rewards:** +10 for timely maintenance, -10 for machine failure

**Policy:** Select the action with the highest Q-value.

##### Python Code



```
Q4_Predictive_Maintenance.py - P:\RL\ASSESSMENT\CO 2\AT1\Q4_Predictive_Maintenance...
File Edit Format Run Options Window Help
import random

states=2
actions=["Maintain","Continue"]
Q=[[0,0] for _ in range(states)]
a,g,e=0.1,0.9,0.2

for _ in range(500):
    s=random.randint(0,1)
    act=random.randint(0,1) if random.random()<e else Q[s].index(max(Q[s]))
    r=10 if (s==1 and act==0) or (s==0 and act==1) else -10
    Q[s][act]+=a*(r+g*max(Q[s])-Q[s][act])
    print("Q Table:",Q)
for i in range(states):
    print("State",i,"->",actions[Q[i].index(max(Q[i]))])

IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\CO 2\AT1\Q4_Predictive_Maintenance.py
Q Table: [[60.1370153454768, 89.15129234922851], [89.47350981516408, 50.91696344764199]]
>>>
State 0 -> Continue
State 1 -> Maintain
>>>
```

##### Code Explanation

**Step 1:** Define two machine states.

- State 0 → Healthy Machine
- State 1 → Faulty Machine

**Step 2:** Define two actions.

- Perform Maintenance
- Continue Operation

**Step 3:** Initialize the Q-table with zeros.

**Step 4:** Train the agent for 500 episodes.

**Step 5:** Select an action using the  $\epsilon$ -greedy strategy.

**Step 6:** Give rewards.

- Correct maintenance decision  $\rightarrow +10$
- Incorrect decision  $\rightarrow -10$

**Step 7:** Update the Q-table using the Q-Learning formula.

**Step 8:** Display the best maintenance policy.

### **Advantages**

- Reduces equipment failures.
- Minimizes maintenance costs.
- Improves machine reliability.
- Learns automatically from experience.

### **Applications**

- Manufacturing industries
- Smart factories
- Power plants
- Industrial robots

### **Conclusion**

Reinforcement Learning improves predictive maintenance by learning optimal maintenance schedules, reducing downtime, and increasing equipment reliability

**Q5. Analyze how Reinforcement Learning can be used to optimize transportation systems such as bus scheduling or route allocation. Define states, actions, and rewards, and discuss how real-time data improves performance. (5 Marks)**

### Aim

To optimize bus scheduling and route allocation using Reinforcement Learning.

### Theory

Reinforcement Learning helps transportation systems improve bus scheduling by analyzing passenger demand and traffic conditions. The agent learns to allocate buses efficiently, reducing waiting time and improving service quality.

### MDP Components

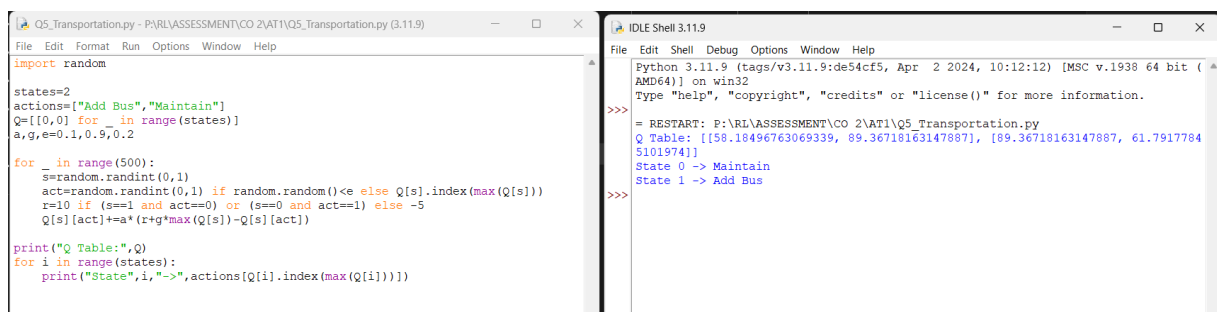
**States:** Low Traffic, High Traffic

**Actions:** Add Bus, Maintain Schedule

**Rewards:** +10 for efficient scheduling, -5 for delays

**Policy:** Select the best scheduling action.

### Python Code



```
import random

states=2
actions=["Add Bus","Maintain"]
Q=[[0,0] for _ in range(states)]
a,g,e=0.1,0.9,0.2

for _ in range(500):
    s=random.randint(0,1)
    act=random.randint(0,1) if random.random()<= e else Q[s].index(max(Q[s]))
    r=10 if (s==1 and act==0) or (s==0 and act==1) else -5
    Q[s][act]+=a*(r+g*max(Q[s])-Q[s][act])

print("Q Table:",Q)
for i in range(states):
    print("State",i,"->",actions[Q[i].index(max(Q[i]))])
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: P:\RL\ASSESSMENT\CO 2\AT1\Q5_Transportation.py
Q Table: [[58.18496763069339, 89.36718163147887], [89.36718163147887, 61.79177845101974]]
State 0 -> Maintain
State 1 -> Add Bus
>>>
```

### Code Explanation

**Step 1:** Define two traffic states.

- State 0 → Low Traffic
- State 1 → High Traffic

**Step 2:** Define two actions.



- Add Bus
- Maintain Schedule

**Step 3:** Initialize the Q-table with zeros.

**Step 4:** Train the agent for 500 episodes.

**Step 5:** Select an action using the  $\epsilon$ -greedy strategy.

**Step 6:** Give rewards.

- Correct scheduling  $\rightarrow +10$
- Incorrect scheduling  $\rightarrow -5$

**Step 7:** Update the Q-table using the Q-Learning formula.

**Step 8:** Display the best bus scheduling policy.

### **Advantages**

- Reduces passenger waiting time.
- Optimizes bus allocation.
- Adapts to traffic changes.
- Improves transport efficiency.

### **Applications**

- Public transportation
- Metro systems
- Smart traffic management
- Logistics routing

### **Conclusion**

RL enables transportation systems to make intelligent scheduling decisions using real-time traffic and passenger information.

## Q6. Apply Reinforcement Learning to dynamic pricing in e-commerce. Define the state space, action space, and reward function. Discuss challenges in balancing exploration and exploitation.

### Aim

To apply Reinforcement Learning for optimizing product prices based on demand.

### Theory

RL learns the best pricing strategy by adjusting prices according to customer demand and sales performance. It balances exploration and exploitation to maximize long-term profit.

### MDP Components

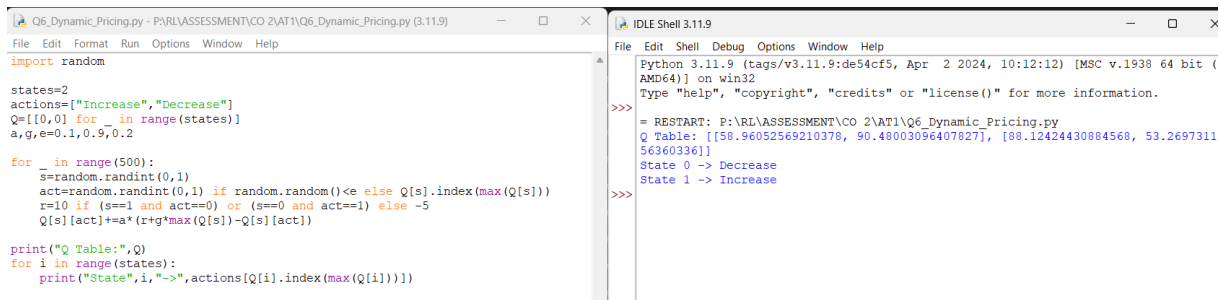
**States:** Low Demand, High Demand

**Actions:** Increase Price, Decrease Price

**Rewards:** +10 for higher profit, -5 for lower sales

**Policy:** Select the price with the highest Q-value.

### Python Code



```
Q6_Dynamic_Pricing.py - P:\RL\ASSESSMENT\CO 2\AT1\Q6_Dynamic_Pricing.py (3.11.9)
File Edit Format Run Options Window Help
import random

states=2
actions=["Increase","Decrease"]
Q=[[0,0] for _ in range(states)]
a,g,e=0.1,0.9,0.2

for _ in range(500):
    s=random.randint(0,1)
    act=random.randint(0,1) if random.random()<e else Q[s].index(max(Q[s]))
    r=10 if (s==1 and act==0) or (s==0 and act==1) else -5
    Q[s][act]+=a*(r+g*max(Q[s])-Q[s][act])

print("Q Table:",Q)
for i in range(states):
    print("State",i,"->",actions[Q[i].index(max(Q[i]))])

IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\CO 2\AT1\Q6_Dynamic_Pricing.py
Q Table: [[58.96052569210378, 90.48003096407827], [88.12424430884568, 53.269731156360336]]
State 0 -> Decrease
State 1 -> Increase
>>>
```

### Code Explanation

**Step 1:** Define two demand states.

- State 0 → Low Demand
- State 1 → High Demand

**Step 2:** Define two actions.

- Increase Price

- Decrease Price

**Step 3:** Initialize the Q-table with zeros.

**Step 4:** Train the agent for 500 episodes.

**Step 5:** Select an action using the  $\epsilon$ -greedy strategy.

**Step 6:** Give rewards.

- Correct pricing decision  $\rightarrow +10$
- Incorrect pricing decision  $\rightarrow -5$

**Step 7:** Update the Q-table using the Q-Learning formula.

**Step 8:** Display the optimal pricing policy.

### **Advantages**

- Maximizes profit.
- Adapts to market demand.
- Learns pricing strategies automatically.
- Improves customer satisfaction.

### **Applications**

- Amazon
- Flipkart
- Airline ticket pricing
- Hotel booking systems

### **Conclusion**

RL enables businesses to determine optimal product prices while balancing revenue and customer demand.

**Q7. Illustrate the use of Reinforcement Learning in smart city waste management systems. Identify the MDP components and discuss how continuous updates improve operational efficiency. (5 Marks)**

**Aim**

To optimize waste collection using Reinforcement Learning.

**Theory**

RL helps waste collection vehicles choose efficient routes based on the fill level of garbage bins. Continuous sensor updates enable timely collection and reduce operational costs.

**MDP Components**

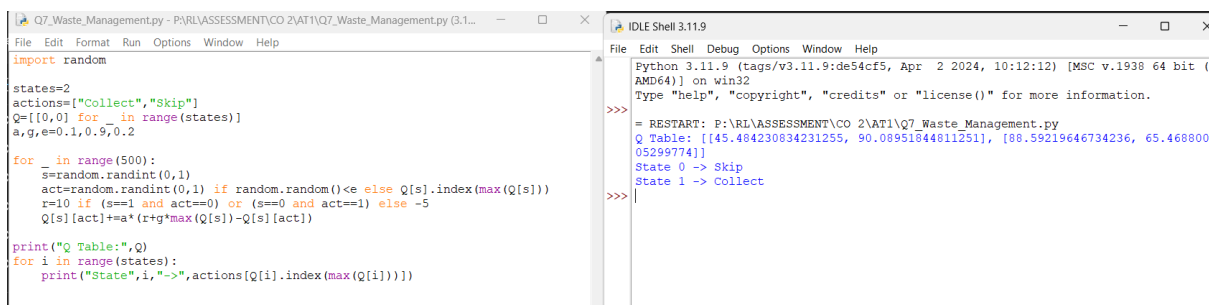
**States:** Bin Empty, Bin Full

**Actions:** Collect Waste, Skip Bin

**Rewards:** +10 for timely collection, -5 for overflow

**Policy:** Select the best collection action.

**Python Code**



```
Q7_Waste_Management.py - P:\RL\ASSESSMENT\CO 2\AT1\Q7_Waste_Management.py (3.1...
File Edit Format Run Options Window Help
import random

states=2
actions=["Collect","Skip"]
Q=[[0,0] for _ in range(states)]
a,g,e=0.1,0.9,0.2

for _ in range(500):
    s=random.randint(0,1)
    act=random.randint(0,1) if random.random()<e else Q[s].index(max(Q[s]))
    r=10 if (s==1 and act==0) or (s==0 and act==1) else -5
    Q[s][act]+=a*(r+g*max(Q[s])-Q[s][act])

print("Q Table:",Q)
for i in range(states):
    print("State",i,"->",actions[Q[i].index(max(Q[i]))])

IDLE Shell 3.11.9
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\CO 2\AT1\Q7_Waste_Management.py
Q Table: [[45.484230834231255, 90.08951844811251], [88.59219646734236, 65.46880005299774]]
State 0 -> Skip
State 1 -> Collect
>>>
```

**Code Explanation**

**Step 1:** Define two bin states.

- State 0 → Bin Empty
- State 1 → Bin Full

**Step 2:** Define two actions.

- Collect Waste
- Skip Bin

**Step 3:** Initialize the Q-table with zeros.

**Step 4:** Train the agent for 500 episodes.

**Step 5:** Select an action using the  $\epsilon$ -greedy strategy.

**Step 6:** Give rewards.

- Timely collection  $\rightarrow +10$
- Wrong decision  $\rightarrow -5$

**Step 7:** Update the Q-table using the Q-Learning formula.

**Step 8:** Display the best waste collection policy.

### **Advantages**

- Reduces fuel consumption.
- Prevents bin overflow.
- Optimizes collection routes.
- Improves city cleanliness.

### **Applications**

- Smart cities
- Municipal waste management
- Recycling systems
- IoT-based garbage monitoring

### **Conclusion**

RL improves waste collection efficiency by learning optimal routes and schedules from continuous environmental updates.

**Q8. Evaluate how Reinforcement Learning can be used in network bandwidth allocation in telecommunications. Define states, actions, and rewards, and examine how the system adapts to dynamic conditions. (5 Marks)**

### Aim

To optimize network bandwidth allocation using Reinforcement Learning.

### Theory

RL dynamically allocates bandwidth according to network traffic and user demand. It continuously learns the best allocation strategy to improve network performance.

### MDP Components

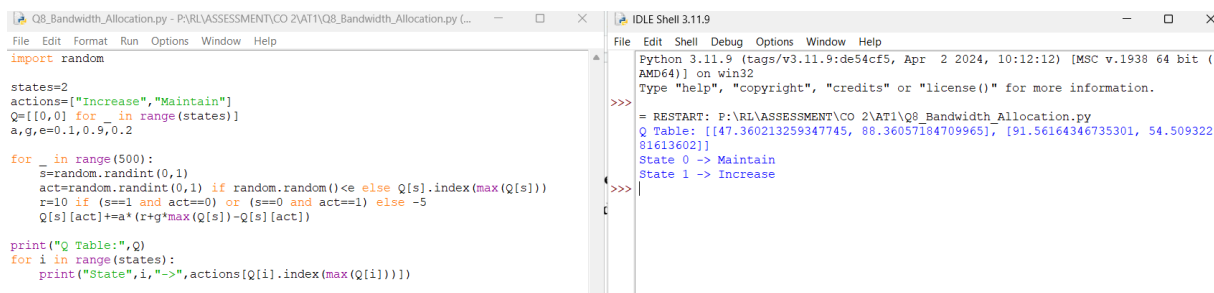
**States:** Low Traffic, High Traffic

**Actions:** Increase Bandwidth, Maintain Bandwidth

**Rewards:** +10 for efficient allocation, -5 for congestion

**Policy:** Select the best bandwidth allocation.

### Python Code



```
Q8_Bandwidth_Allocation.py - PY\RL\ASSESSMENT\CO 2\AT1\Q8_Bandwidth_Allocation.py (...)
```

```
import random

states=2
actions=["Increase","Maintain"]
Q=[[0,0] for _ in range(states)]
a,g,e=0.1,0.9,0.2

for _ in range(500):
    s=random.randint(0,1)
    act=random.randint(0,1) if random.random()<e else Q[s].index(max(Q[s]))
    r=10 if (s==1 and act==0) or (s==0 and act==1) else -5
    Q[s][act]+=a*(r+g*max(Q[s])-Q[s][act])

print("Q Table:",Q)
for i in range(states):
    print("State",i,"->",actions[Q[i].index(max(Q[i]))])
```

```
IDLE Shell 3.11.9
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\CO 2\AT1\Q8_Bandwidth_Allocation.py
Q Table: [[47.360213259347745, 88.36057184709965], [91.56164346735301, 54.50932281613602]]
State 0 -> Maintain
State 1 -> Increase
>>>
```

### Code Explanation

**Step 1:** Define two network states.

- State 0 → Low Traffic
- State 1 → High Traffic

**Step 2:** Define two actions.

- Increase Bandwidth
- Maintain Bandwidth

**Step 3:** Initialize the Q-table with zeros.

**Step 4:** Train the agent for 500 episodes.

**Step 5:** Select an action using the  $\epsilon$ -greedy strategy.

**Step 6:** Give rewards.

- Efficient bandwidth allocation  $\rightarrow +10$
- Poor allocation  $\rightarrow -5$

**Step 7:** Update the Q-table using the Q-Learning formula.

**Step 8:** Display the optimal bandwidth allocation policy.

### **Advantages**

- Reduces network congestion.
- Improves bandwidth utilization.
- Adapts to changing traffic.
- Enhances user experience.

### **Applications**

- Mobile networks
- Wi-Fi systems
- Internet service providers
- Cloud computing

### **Conclusion**

RL enables efficient bandwidth allocation by adapting to changing network conditions in real time.

**Q9. Analyze the application of Reinforcement Learning in portfolio management in finance. Discuss advantages over traditional models and challenges such as risk and delayed rewards. (5 Marks)**

**Aim**

To optimize investment decisions using Reinforcement Learning.

**Theory**

RL assists investors by learning investment strategies from market conditions. It maximizes long-term returns while managing financial risks through continuous learning.

**MDP Components**

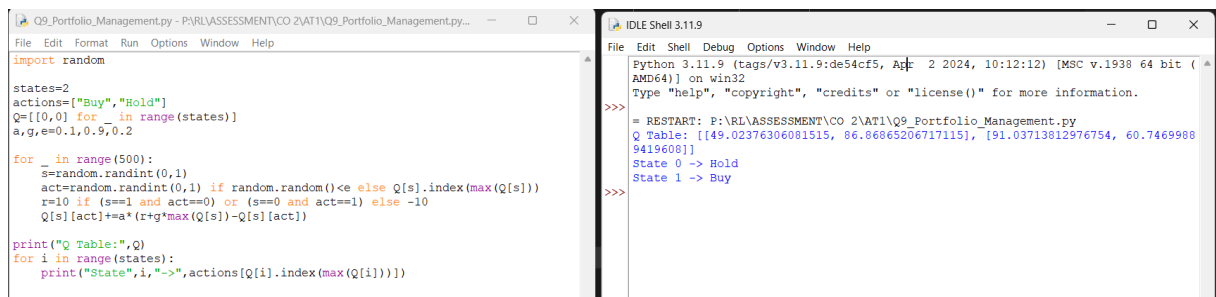
**States:** Market Up, Market Down

**Actions:** Buy, Hold

**Rewards:** +10 for profit, -10 for loss

**Policy:** Select the action with the highest Q-value.

**Python Code**



The image shows a screenshot of a Python script and its execution in IDLE Shell. The script, named 'Q9\_Portfolio\_Management.py', defines two states (0 and 1) and two actions ('Buy' and 'Hold'). It uses a Q-table to store the expected returns for each state-action pair. The script simulates 500 episodes, updating the Q-table based on the rewards received. The execution output shows the Q-table values and the actions selected for each state.

```
import random

states=2
actions=["Buy","Hold"]
Q=[0,0] for _ in range(states)
a,g,e=0.1,0.9,0.2

for _ in range(500):
    s=random.randint(0,1)
    act=random.randint(0,1) if random.random()<e else Q[s].index(max(Q[s]))
    r=10 if (s==1 and act==0) or (s==0 and act==1) else -10
    Q[s][act]+=a*(r+g*max(Q[s])-Q[s][act])

print("Q Table:",Q)
for i in range(states):
    print("State",i,"->",actions[Q[i].index(max(Q[i]))])
```

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>> = RESTART: P:\RL\ASSESSMENT\CO 2\AT1\Q9\_Portfolio\_Management.py  
Q Table: [[49.02376306081515, 86.86865206717115], [91.03713812976754, 60.74699889419608]]  
State 0 -> Hold  
State 1 -> Buy  
>>>

**Code Explanation**

**Step 1:** Define two market states.

- State 0 → Market Down
- State 1 → Market Up

**Step 2:** Define two actions.

- Buy



- Hold

**Step 3:** Initialize the Q-table with zeros.

**Step 4:** Train the agent for 500 episodes.

**Step 5:** Select an action using the  $\epsilon$ -greedy strategy.

**Step 6:** Give rewards.

- Profitable investment  $\rightarrow +10$
- Loss-making decision  $\rightarrow -10$

**Step 7:** Update the Q-table using the Q-Learning formula.

**Step 8:** Display the best investment policy.

### **Advantages**

- Learns market behavior.
- Optimizes investment returns.
- Adapts to market changes.
- Supports automated trading.

### **Applications**

- Stock trading
- Mutual funds
- Investment advisory systems
- Financial analytics

### **Conclusion**

RL provides intelligent portfolio management by learning investment strategies that maximize long-term profit while handling market uncertainty

**Q10. Justify the use of Reinforcement Learning in renewable energy management systems. Define the RL framework components and discuss the impact of environmental uncertainty on decision-making.**

### Aim

To optimize renewable energy usage using Reinforcement Learning.

### Theory

RL manages renewable energy resources by deciding when to store, use, or distribute electricity based on production and demand. It adapts to changing weather and energy consumption patterns.

### MDP Components

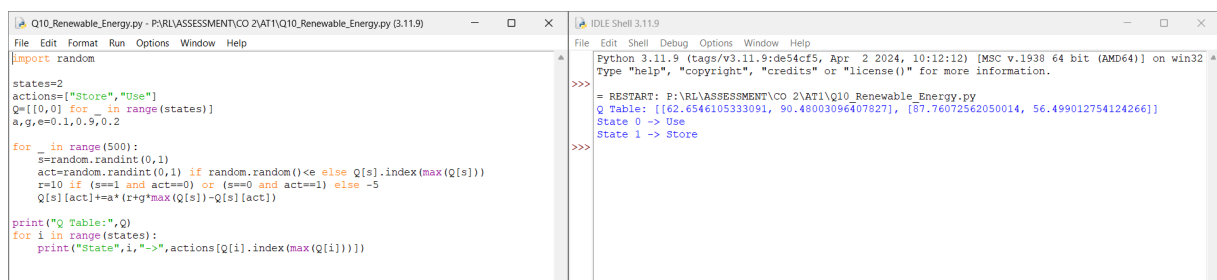
**States:** Low Energy, High Energy

**Actions:** Store Energy, Use Energy

**Rewards:** +10 for efficient utilization, -5 for energy waste

**Policy:** Select the action with the highest Q-value.

### Python Code



```
Q10_Renewable_Energy.py - P:\RL\ASSESSMENT\CO 2\AT1\Q10_Renewable_Energy.py (3.11.9)
File Edit Format Run Options Window Help
import random

states=2
actions=["Store","Use"]
Q=[[0,0] for _ in range(states)]
a,g,e=0,1,0,0.2

for _ in range(500):
    s=random.randint(0,1)
    act=random.randint(0,1) if random.random()<e else Q[s].index(max(Q[s]))
    r=10 if (s==1 and act==0) or (s==0 and act==1) else -5
    Q[s][act]+=a*(r+g*max(Q[s])-Q[s][act])

print("Q Table:",Q)
for i in range(states):
    print("State",i,"->",actions[Q[i].index(max(Q[i]))])

IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:\RL\ASSESSMENT\CO 2\AT1\Q10_Renewable_Energy.py
Q Table: [[62.6546105333091, 90.48003096407827], [87.76072562050014, 56.499012754124266]]
State 0 -> Use
State 1 -> Store
>>>
```

### Code Explanation

**Step 1:** Define two energy states.

- State 0 → Low Energy
- State 1 → High Energy

**Step 2:** Define two actions.

- Store Energy
- Use Energy

**Step 3:** Initialize the Q-table with zeros.

**Step 4:** Train the agent for 500 episodes.

**Step 5:** Select an action using the  $\epsilon$ -greedy strategy.

**Step 6:** Give rewards.

- Efficient energy management  $\rightarrow +10$
- Energy wastage  $\rightarrow -5$

**Step 7:** Update the Q-table using the Q-Learning formula.

**Step 8:** Display the optimal energy management policy.

### **Advantages**

- Reduces energy waste.
- Improves battery utilization.
- Adapts to weather changes.
- Supports sustainable energy management.

### **Applications**

- Smart grids
- Solar power systems
- Wind energy farms
- Battery storage systems

### **Conclusion**

Reinforcement Learning improves renewable energy management by learning optimal decisions under uncertain environmental conditions, leading to efficient and sustainable energy utilization.